Databases

Data, Guarantees, and Correctness in Financial Systems

FinTech 512 – Software Engineering for Financial Technology

Duke

What have we seen so far?

Files: CSV

SPY.csv 503.77 KiB

Date	Open	High	Low	Close	Adj Close	Volume
1993-01-29	43.968750	43.968750	43.750000	43.937500	25.968958	1003200
1993-02-01	43.968750	44.250000	43.968750	44.250000	26.153660	480500
1993-02-02	44.218750	44.375000	44.125000	44.343750	26.209057	201300
1993-02-03	44.406250	44.843750	44.375000	44.812500	26.486113	529400
1993-02-04	44.968750	45.093750	44.468750	45.000000	26.596937	531500
1993-02-05	44.968750	45.062500	44.718750	44.968750	26.578478	492100
1993-02-08	44.968750	45.125000	44.906250	44.968750	26.578478	596100
1993-02-09	44.812500	44.812500	44.562500	44.656250	26.393753	122100
1993-02-10	44.656250	44.750000	44.531250	44.718750	26.430716	379600
1993-02-11	44.781250	45.125000	44.781250	44.937500	26.559996	19500
1993-02-12	44.875000	44.875000	44.593750	44.593750	26.356831	42500

Files: JavaScript Object Notation (JSON)

```
{
    "Image": {
        "Width": 800,
        "Height": 600,
        "Title": "View from 15th Floor",
        "Thumbnail": {
            "Url": "http://www.example.com/image/481989943",
            "Height": 125,
            "Width": 100
        },
        "Animated" : false,
        "IDs": [116, 943, 234, 38793]
    }
}
```

Resources:

- https://json.org
- https://jsonlint.com
- https://realpython.com/python-json/

https://www.rfc-editor.org/rfc/rfc7159#section-13

Files Are Data, Not Systems

Files (CSV, JSON, text) are useful, but they do not provide:

- Concurrency control
- Atomic updates
- Integrity enforcement
- Crash recovery
- Access coordination

- What Goes Wrong with Files
  - Two writers overwrite each other
  - Partial writes leave corrupted state
  - Invariants must be enforced manually
  - Recovery requires custom logic

Databases Are Systems, Not Storage

Databases are not just places to store data.

They are systems that enforce **correctness** under **concurrency and failure**.

What Databases Do That Files Don't

- Coordinate concurrent access
- Enforce integrity constraints
- Prevent partial updates
- Recover from crashes
- Preserve correctness over time
- Enable authentication and authorization

Online Stock Trading Platform

Consider what happens when …

Alice sells 100 shares of AAPL to Bob.

The system must:

- debit her account by 100 shares
- credit Bob's account with 100 shares
- debit Bob's account by the correct amount of cash
- credit Alice's account with the proceeds
- apply any commission(s) to both Alice and Bob.

Partial Failure Is Catastrophic

In real systems, failures happen:

- Crashes
- Network timeouts
- Power loss
- Concurrent updates

The question is **not whether failure occurs**, but **what state is left behind**.

Why Partial Failure Is Dangerous

- Some updates succeed, others don't
- The system enters an **inconsistent state**
- Recovery is unclear or impossible

Transactions as Units of Work

Transaction: a group of operations treated as a **single unit of work**.

- Either all operations succeed

or -

- None of them do

Transactions ensure that:

- Partial updates do not escape
- Failures do not leave an inconsistent state
- Recovery is well-defined

Example: Money Transfer

A single transaction includes:

- Debit Account A
- Credit Account B
- Record the transfer

If any step fails: No step is applied

ACID Properties (Formal Definitions)

Atomicity

- A transaction's operations are **all applied or none are applied**
- No partial effects are visible

Consistency

- A transaction moves the database from one **valid state** to another
- All defined constraints are preserved

Isolation

- Concurrent transactions appear to execute **as if run one at a time**
- Intermediate states are not visible to others

Durability

- Once a transaction commits, its effects **persist**, even after crashes

ACID Reframed: Failures Prevented

Why Each ACID Property Exists

Each ACID property prevents a **specific class of failure** that occurs in real systems.

Atomicity

Prevents:

- Partial financial effects
- One-sided updates

Ex:Debit succeeds, credit fails → not allowed

Consistency

Prevents:

- Violated business rules
- Broken invariants

Ex: Negative account balances

Isolation

Prevents:

- Dirty reads
- Lost updates
- Inconsistent summaries

Example:
Two concurrent visits increment the same counter incorrectly

Durability

Prevents:

- “It worked, but then we crashed”
- Lost committed decisions

Example:
Audit record disappears after restart

Concurrency Anomalies

In shared systems, multiple transactions run **at the same time**.
Even correct logic can fail when operations **interleave**.

Common Concurrency Anomalies

- **Lost Update**
  - Two transactions overwrite each other's changes
- **Dirty Read**
  - A transaction sees uncommitted data
- **Non-Repeatable Read**
  - A value changes mid-transaction

Concrete Examples

- **Banking**
  - Two transfers read the same balance and both approve
- **URL Shortener**
  - Two visits increment the same counter incorrectly
  - An aggregate reflects fewer visits than actually occurred

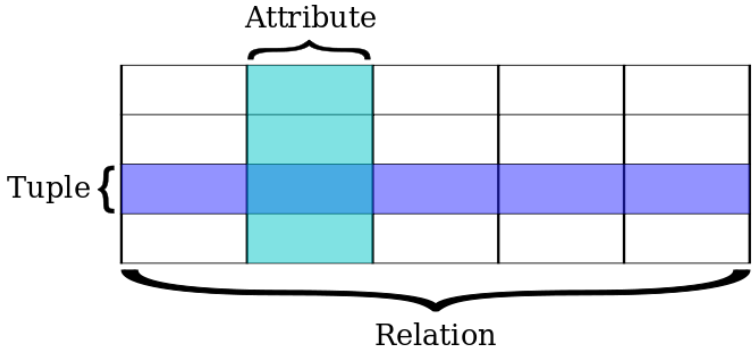
Data Models: Organizing Data

What Is a Data Model?

- what data exists
- how data is structured
- how pieces of data relate to each other
- what rules must always hold

The Relational Data Model

- Data is organized into **tables**
- Each table represents **one kind of thing**
- Each row represents **one instance**
- Each column represents **a property of that instance**



Row	Tuple / Record	Data set representing a single item
Column	Attribute / field	Labeled element of a record
Table	Relation	Set of records with the same attributes

Tables Represent Kinds of Things

Consider an online music store –

- chinook (SQLite 3)
  - Tables (12)
    - albums
    - artists
    - customers
    - employees
    - genres
    - invoice_items
    - invoices
    - media_types
    - playlist_track
    - playlists
    - stock_data
    - tracks

chinook | Table name: customers | WITHOUT ROWID | STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	Default value
1	CustomerId	INTEGER								NULL
2	FirstName	NVARCHAR (40)								NULL
3	LastName	NVARCHAR (20)								NULL
4	Company	NVARCHAR (80)								NULL
5	Address	NVARCHAR (70)								NULL
6	City	NVARCHAR (40)								NULL
7	State	NVARCHAR (40)								NULL
8	Country	NVARCHAR (40)								NULL
9	PostalCode	NVARCHAR (10)								NULL
10	Phone	NVARCHAR (24)								NULL
11	Fax	NVARCHAR (24)								NULL
12	Email	NVARCHAR (60)								NULL
13	SupportRepId	INTEGER								NULL

chinook | Table name: invoices | WITHOUT ROWID | STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	Default value
1	InvoiceId	INTEGER								NULL
2	CustomerId	INTEGER								NULL
3	InvoiceDate	DATETIME								NULL
4	BillingAddress	NVARCHAR (70)								NULL
5	BillingCity	NVARCHAR (40)								NULL
6	BillingState	NVARCHAR (40)								NULL
7	BillingCountry	NVARCHAR (40)								NULL
8	BillingPostalCode	NVARCHAR (10)								NULL
9	Total	NUMERIC (10,2)								NULL

Tables Contain Records (Facts)

Customers and Invoices in an Online Music Store

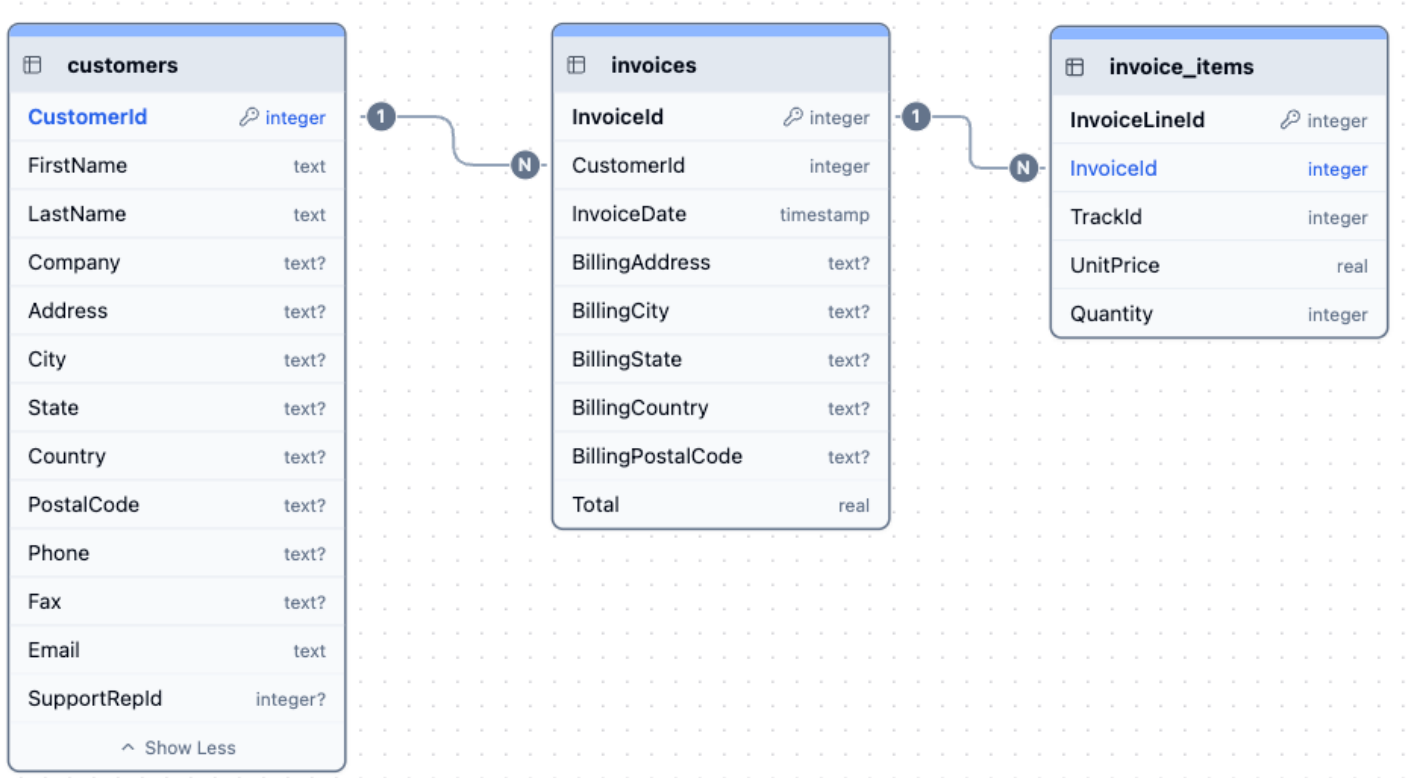
	CustomerId	FirstName	LastName	Company	Address	City	State	Country	PostalCode	Phone	Fax	Email	SupportRepId
1	1	Luís	Gonçalves	Embraer - Empresa ...	Av.Brigadeiro Faria Lima,2170	São José dos Campos	SP	Brazil	12227-000	+55 (12) 3923-5555	+55 (12) 3923-5566	luisg@embraer.com.br	3
2	2	Leonie	Köhler	NULL	Theodor-Heuss-Straße 34	Stuttgart	NULL	Germany	70174	+49 0711 2842222	NULL	leonekohler@surfeu.de	5
3	3	François	Tremblay	NULL	1498 rue Bélanger	Montréal	QC	Canada	H2G 1A7	+1 (514) 721-4711	NULL	ftremblay@gmail.com	3
4	4	Bjørn	Hansen	NULL	Ullevålsveien 14	Oslo	NULL	Norway	0171	+47 22 44 22 22	NULL	bjorn.hansen@yahoo.no	4
5	5	František	Wichterlová	JetBrains s.r.o.	Klanova 9/506	Prague	NULL	Czech Republic	14700	+420 2 4172 5555	+420 2 4172 5555	frantisekw@jetbrains.com	4
6	6	Helena	Holý	NULL	Rilská 3174/6	Prague	NULL	Czech Republic	14300	+420 2 4177 0449	NULL	hholy@gmail.com	5
7	7	John	Slankas	NULL	Rotenturmstraße 4,1010 Innere Stadt	Vienne	NULL	Austria	1010	+43 01 5134505	NULL	astrid.gruber@apple.at	5
8	8	Daan	Peeters	NULL	Grétrystraat 63	Brussels	NULL	Belgium	1000	+32 02 219 03 03	NULL	daan_peeters@apple.be	4
9	9	Kara	Nielsen	NULL	Sønder Boulevard 51	Copenhagen	NULL	Denmark	1720	+453 3331 9991	NULL	kara.nielsen@jubii.dk	4
10	10	Eduardo	Martins	Woodstock Discos	Rua Dr.Falcão Filho,155	São Paulo	SP	Brazil	01007-010	+55 (11) 3033-5446	+55 (11) 3033-4564	eduardo@woodstock.com.br	4
11	11	Alexandre	Rocha	Banco do Brasil S.A.	Av.Paulista,2022	São Paulo	SP	Brazil	01310-200	+55 (11) 3055-3278	+55 (11) 3055-8131	alero@uol.com.br	5
12	12	Roberto	Almeida	Riotur	Praça Pio X,119	Rio de Janeiro	RJ	Brazil	20040-020	+55 (21) 2271-7000	+55 (21) 2271-7070	roberto.almeida@riotur.gov.br	3
13	13	Fernanda	Ramos	NULL	Qe 7 Bloco G	Brasília	DF	Brazil	71020-677	+55 (61) 3363-5547	+55 (61) 3363-7855	fernadaramos4@uol.com.br	4
14	14	Mark	Philips	Telus	8210 111 ST NW	Edmonton	AB	Canada	T6G 2C7	+1 (780) 434-4554	+1 (780) 434-5565	mphilips12@shaw.ca	5
15	15	Jennifer	Peterson	Rogers Canada	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 (604) 688-8756	jenniferp@rogers.ca	3
16	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	+1 (650) 253-0000	+1 (650) 253-0000	fharris@google.com	4

	InvoiceIc	Custome	InvoiceDate	BillingAddress	BillingCity	BillingSt:	BillingCountry	BillingPostalC	Total
1	1	2	2009-01-01 00:00:00	Theodor-Heuss-Straße 34	Stuttgart	NULL	Germany	70174	1.98
2	2	4	2009-01-02 00:00:00	Ullevålsveien 14	Oslo	NULL	Norway	0171	3.96
3	3	8	2009-01-03 00:00:00	Grétrystraat 63	Brussels	NULL	Belgium	1000	5.94
4	4	14	2009-01-06 00:00:00	8210 111 ST NW	Edmonton	AB	Canada	T6G 2C7	8.91
5	5	23	2009-01-11 00:00:00	69 Salem Street	Boston	MA	USA	2113	13.86
6	6	37	2009-01-19 00:00:00	Berger Straße 10	Frankfurt	NULL	Germany	60316	0.99
7	7	38	2009-02-01 00:00:00	Barbarossastraße 19	Berlin	NULL	Germany	10779	1.98
8	8	40	2009-02-01 00:00:00	8,Rue Hanovre	Paris	NULL	France	75002	1.98
9	9	42	2009-02-02 00:00:00	9,Place Louis Barthou	Bordeaux	NULL	France	33000	3.96
10	10	46	2009-02-03 00:00:00	3 Chatham Street	Dublin	Dublin	Ireland	NULL	5.94
11	11	52	2009-02-06 00:00:00	202 Hoxton Street	London	NULL	United Kingdom	N1 5LH	8.91

Relationships Between Tables

Tables do not exist in isolation

- Relationships connect facts across tables
- Relationships preserve meaning without duplication
- Databases enforce relationships consistently



Duke

Without Separate Tables

Refresh table data (F5)

	Custome	FirstName	LastName	Company	Address	City	State	Country	PostalCode	Phone	Fax	Email	SupportR	InvoiceIc	Custome	InvoiceDate	BillingAddress	BillingCity	BillingSt	BillingCountry	BillingPostalC	Total	InvoiceLi	InvoiceIc	TrackId	UnitPrice	Quantity
553	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	102	15	2010-03-16 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	9.91	549	102	3314	0.99	1
554	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	102	15	2010-03-16 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	9.91	550	102	3320	0.99	1
555	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	102	15	2010-03-16 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	9.91	551	102	3326	0.99	1
556	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	102	15	2010-03-16 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	9.91	552	102	3332	0.99	1
557	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	102	15	2010-03-16 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	9.91	553	102	3338	1.99	1
558	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	231	15	2011-10-21 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	1.98	1253	231	649	0.99	1
559	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	231	15	2011-10-21 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	1.98	1254	231	650	0.99	1
560	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	254	15	2012-01-23 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	3.96	1371	254	1352	0.99	1
561	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	254	15	2012-01-23 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	3.96	1372	254	1354	0.99	1
562	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	254	15	2012-01-23 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	3.96	1373	254	1356	0.99	1
563	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	254	15	2012-01-23 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	3.96	1374	254	1358	0.99	1
564	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	276	15	2012-04-26 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	5.94	1489	276	2058	0.99	1
565	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	276	15	2012-04-26 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	5.94	1490	276	2062	0.99	1
566	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	276	15	2012-04-26 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	5.94	1491	276	2066	0.99	1
567	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	276	15	2012-04-26 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	5.94	1492	276	2070	0.99	1
568	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	276	15	2012-04-26 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	5.94	1493	276	2074	0.99	1
569	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	276	15	2012-04-26 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	5.94	1494	276	2078	0.99	1
570	15	Jennifer	Peterson	Rogers ...	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	+1 (604) 688-2255	+1 ...	jenniferp@rogers.ca	3	328	15	2012-12-15 00:00:00	700 W Pender Street	Vancouver	BC	Canada	V6C 1G8	0.99	1784	328	393	0.99	1
571	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	13	16	2009-02-19 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	0.99	74	13	462	0.99	1
572	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	134	16	2010-08-13 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	1.98	723	134	907	0.99	1
573	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	134	16	2010-08-13 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	1.98	724	134	909	0.99	1
574	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	145	16	2010-09-23 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	13.86	782	145	1236	0.99	1
575	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	145	16	2010-09-23 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	13.86	783	145	1245	0.99	1
576	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	145	16	2010-09-23 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	13.86	784	145	1254	0.99	1
577	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	145	16	2010-09-23 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	13.86	785	145	1263	0.99	1
578	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	145	16	2010-09-23 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	13.86	786	145	1272	0.99	1
579	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	145	16	2010-09-23 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	13.86	787	145	1281	0.99	1
580	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	145	16	2010-09-23 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	13.86	788	145	1290	0.99	1
581	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	145	16	2010-09-23 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	13.86	789	145	1299	0.99	1
582	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	145	16	2010-09-23 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	13.86	790	145	1308	0.99	1
583	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	145	16	2010-09-23 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	13.86	791	145	1317	0.99	1
584	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	145	16	2010-09-23 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	13.86	792	145	1326	0.99	1
585	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	145	16	2010-09-23 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	13.86	793	145	1335	0.99	1
586	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	145	16	2010-09-23 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	13.86	794	145	1344	0.99	1
587	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	145	16	2010-09-23 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	13.86	795	145	1353	0.99	1
588	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	200	16	2011-05-24 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	8.91	1077	200	3035	0.99	1
589	16	Frank	Harris	Google Inc.	1600 Amphitheatre Parkway	Mountain ...	CA	USA	94043-1351	+1 (650) 253-0000	+1 ...	fharris@google.com	4	200	16	2011-05-24 00:00:00	1600 Amphitheatre Parkway	Mountain View	CA	USA	94043-1351	8.91	1078	200	3041	0.99	1

Asking Questions of the Database

- Databases are used by **asking questions**
- Questions describe **what information we want**
- Questions can involve **one table or many**
- The database answers based on the data it stores

Which customers bought music?

```
SELECT c.FirstName || ' ' || c.LastName AS CustomerName,
    SUM(i.Total) AS TotalSpent
FROM customers c
JOIN invoices i ON c.CustomerId = i.CustomerId
GROUP BY c.CustomerId, c.FirstName, c.LastName
ORDER BY TotalSpent DESC;
```

	CustomerName	TotalSpent
1	Helena Holý	49.62
2	Richard Cunningham	47.62
3	Luis Rojas	46.62
4	Ladislav Kovács	45.62
5	Hugh O'Reilly	45.62
6	Frank Ralston	43.62
7	Julia Barnett	43.62
8	Fynn Zimmermann	43.62
9	John Slankas	42.62
10	Victor Stevens	42.62

Reconstructing Meaning with Queries

- Information is distributed across tables
  - Queries combine related data
  - Joins reconstruct meaningful views
  - The database decides how to execute the query

SQL as a Declarative Language

- You specify what data you want
  - You do not specify how to retrieve it
  - The database is responsible for correctness
  - Queries are part of the system contract

Querying a Single Table

```
SELECT FirstName, LastName, State
FROM customers
```

	FirstName	LastName	State
14	Mark	Philips	AB
15	Jennifer	Peterson	BC
16	Frank	Harris	CA
17	Jack	Smith	WA
18	Michelle	Brooks	NY
19	Tim	Goyer	CA
20	Dan	Miller	CA
21	Kathy	Chase	NV
22	Heather	Leacock	FL

```
SELECT CustomerId,email
  FROM Customers
```

	CustomerId	Email
1	1	luisg@embraer.com.br
2	2	leonekohler@surfeu.de
3	3	ftremblay@gmail.com
4	4	bjorn.hansen@yahoo.no
5	5	frantisekw@jetbrains.com
6	6	hholy@gmail.com
7	7	astrid.gruber@apple.at
8	8	daan_peeters@apple.be
9	9	kara.nielsen@jubii.dk
10	10	eduardo@woodstock.co...
11	11	alero@uol.com.br
12	12	roberto.almeida@riotur....

Filtering Rows

```
SELECT FirstName, LastName, Country
  FROM Customers
  WHERE Country = 'USA'
```

	FirstName	LastName	Country
1	Frank	Harris	USA
2	Jack	Smith	USA
3	Michelle	Brooks	USA
4	Tim	Goyer	USA
5	Dan	Miller	USA
6	Kathy	Chase	USA
7	Heather	Leacock	USA
8	John	Gordon	USA
9	Frank	Ralston	USA
10	Victor	Stevens	USA
11	Richard	Cunningham	USA
12	Patrick	Gray	USA
13	Julia	Barnett	USA

Ordering Results

```
SELECT FirstName, LastName, Country
  FROM Customers
  WHERE Country = 'USA’
  ORDER BY LastName desc
```

	FirstName	LastName	Country
1	Victor	Stevens	USA
2	Jack	Smith	USA
3	Frank	Ralston	USA
4	Dan	Miller	USA
5	Heather	Leacock	USA
6	Frank	Harris	USA
7	Patrick	Gray	USA
8	Tim	Goyer	USA
9	John	Gordon	USA
10	Richard	Cunningham	USA
11	Kathy	Chase	USA
12	Michelle	Brooks	USA
13	Julia	Barnett	USA

Summarizing Results

```
SELECT Country, COUNT(*) AS NumCustomers
  FROM Customers
  GROUP BY Country
  ORDER BY NumCustomers desc;
```

	Country	NumCustomers
1	USA	13
2	Canada	8
3	France	5
4	Brazil	5
5	Germany	4
6	United Kingdom	3
7	Portugal	2
8	India	2
9	Czech Republic	2
10	Sweden	1
11	Spain	1

Joining Tables

```
SELECT c.FirstName, c.LastName,
       i.InvoiceDate, i.Total
  FROM Customers c
  JOIN Invoices I
    ON c.CustomerId = i.CustomerId
  ORDER BY i.Total desc
```

	FirstName	LastName	InvoiceDate	Total
1	Helena	Holý	2013-11-13 00:00:00	25.86
2	Richard	Cunningham	2012-08-05 00:00:00	23.86
3	Ladislav	Kovács	2010-02-18 00:00:00	21.86
4	Hugh	O'Reilly	2011-04-28 00:00:00	21.86
5	John	Stevens	2010-01-18 00:00:00	18.86
6	Victor	Stevens	2011-05-29 00:00:00	18.86
7	Luis	Rojas	2010-01-13 00:00:00	17.91
8	František	Wichterlová	2012-09-05 00:00:00	16.86
9	Isabelle	Mercier	2012-10-06 00:00:00	16.86
10	Bjørn	Hansen	2011-06-29 00:00:00	15.86

Filtered Joined Results

```
SELECT c.FirstName, c.LastName,
       i.InvoiceDate, i.Total
  FROM Customers c
  JOIN Invoices i
    ON c.CustomerId = i.CustomerId
  WHERE i.total < 10.00
  ORDER BY i.Total desc
```

	FirstName	LastName	InvoiceDate	Total
1	Jennifer	Peterson	2010-03-16 00:00:00	9.91
2	Johannes	Van der Berg	2011-06-21 00:00:00	8.94
3	Luís	Gonçalves	2013-08-07 00:00:00	8.91
4	Leonie	Köhler	2009-10-12 00:00:00	8.91
5	François	Tremblay	2010-12-20 00:00:00	8.91
6	Bjørn	Hansen	2012-02-27 00:00:00	8.91
7	František	Wichterlová	2013-05-06 00:00:00	8.91
8	Helena	Holý	2009-07-11 00:00:00	8.91
9	John	Stevens	2010-09-18 00:00:00	8.91
10	Daan	Peeters	2011-11-26 00:00:00	8.91

Inserting Data

```
INSERT INTO Customer
    (CustomerId,FirstName, LastName, Email)
  VALUES
    (1, 'Jane','Doe','jane.doe@example.com')
```

Single-Table Integrity Constraints

- Rules that **must always hold** for a table
- Enforced **by the database**, not application code
- Prevent invalid data from being stored
- Checked on **INSERT** and **UPDATE**

```
CREATE TABLE Customer (
    CustomerId INTEGER PRIMARY KEY,
    FirstName  TEXT NOT NULL,
    LastName   TEXT NOT NULL,
    Email      TEXT NOT NULL
);
```

Data Definition Language

When a Constraint Is Violated

Example: Missing Required Data

```
INSERT INTO Customer
  (CustomerId, FirstName, LastName)
VALUES
  (2, 'Jane','Doe');
```

Result:
INSERT fails — `Email` cannot be NULL

Identifying Rows: Keys

- Tables contain **many rows**
- Each row must be **uniquely identifiable**
- A **key** is a column (or set of columns) that uniquely identifies a row
- Keys let the database refer to **one specific fact**

Primary Key

- The **main identifier** for a row
- Exactly one per table
- Must be **unique** and **not null**
- Chosen as the official way to refer to a row

Alternate Keys

- Other columns that are **also unique**
- Can identify a row, but are **not the primary choice**
- Still enforced by the database

Integrity Constraints Across Tables

- Tables are related through **keys**
- The database must ensure those relationships remain valid
- Cross-table constraints prevent **dangling or inconsistent references**

Foreign Keys

- A **foreign key** references a primary (or alternate) key in another table
- Ensures referenced rows **actually exist**
- Enforced on **INSERT**, **UPDATE**, and **DELETE**

Invoice.CustomerId → Customer.CustomerId

Why use a DBMS?

- Data Independence
- Efficient Data Access
- Data Integrity and Security
- Data Administration
- Concurrent Access and Crash Recovery
- Reduced Application Development Time

Using LLMs with Databases

What LLMs Are Good At

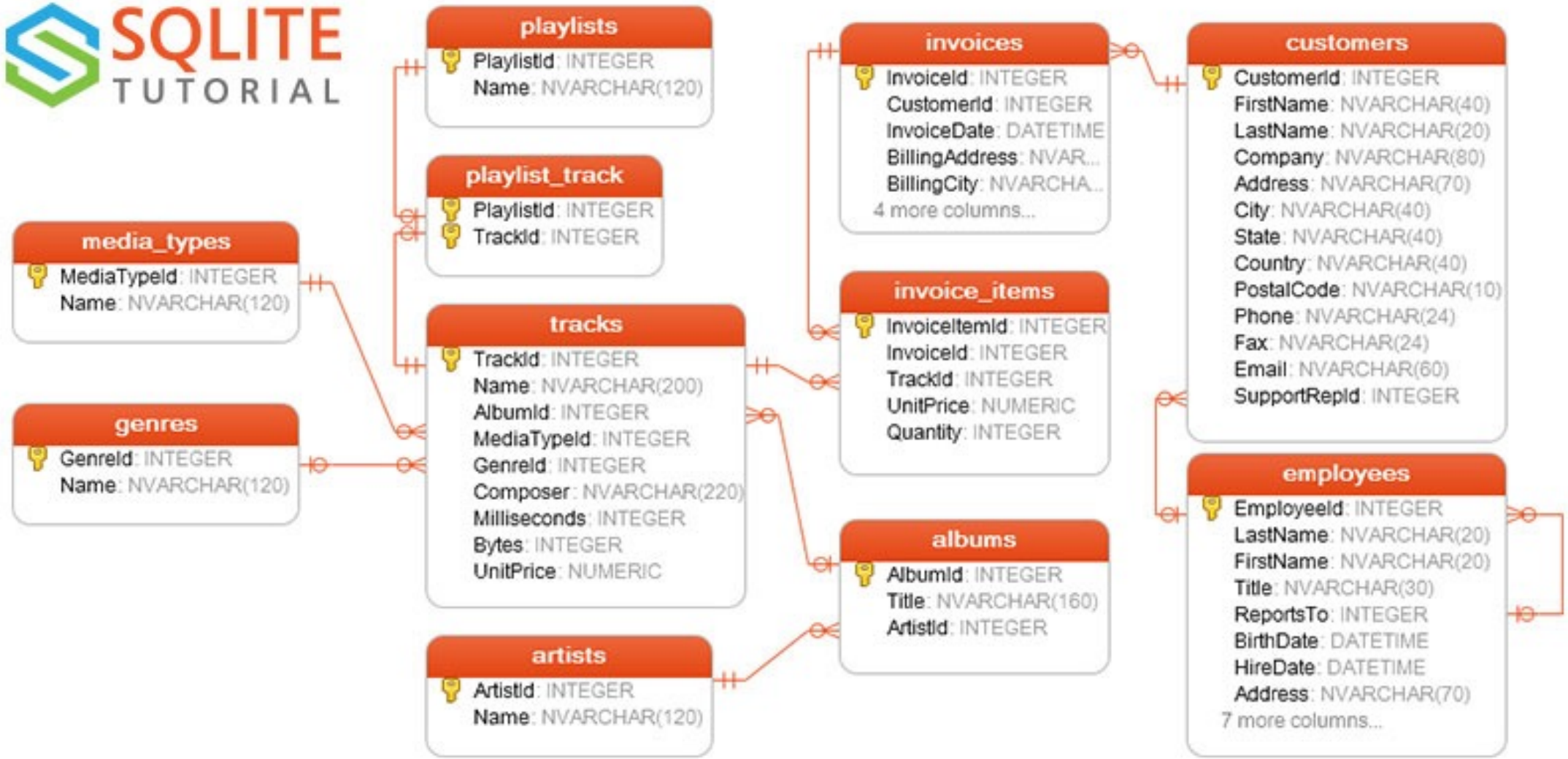
- Translating natural language into queries
- Exploring unfamiliar schemas
- Drafting SQL for review
- Generating test data and edge cases
- Explaining query intent

What LLMs Must *Not* Be Trusted With

- Enforcing correctness
- Preserving invariants
- Deciding what data is valid
- Executing unchecked writes
- Replacing constraints or transactions

Reference Slides

Managing an Online Music Store: Chinook



Basic Query Structure: select

SELECT $A_1, A_2, \ldots, A_n$
FROM $r_1, r_2, \ldots, r_m$
WHERE P

- A_i represents an attribute / column
- R_i represents a relation / table
- P is a predicate.

Note: the result of a select query is a relation

Select: Unique Values

```
SELECT DISTINCT state
  FROM customers
```

Select: Multiple tables

```
SELECT a.name as ArtistName, b.title
   FROM artists  a
   INNER JOIN albums b ON a.ArtistId = b.ArtistId
   WHERE a.name like 'A%’ AND
         b.title like 'T%’

SELECT a.name as ArtistName, b.title
   FROM artists  a, albums b
   WHERE a.ArtistId = b.ArtistId AND
         a.name like 'A%’ AND
         b.title like 'T%'
```

Select: Ordering Results

```
select *
  from invoices
  order by total desc
```

```
select *
  from invoices
  order by billingState,  total desc
```

Aggregate Functions

avg: average value
min: minimum value
max: maximum value
sum: sum of values
count: number of values

```
SELECT count(total), min(total), max(total),
       avg(total), sum(total)
    FROM invoices
    WHERE BillingCountry != 'USA'
```

Aggregate Functions: Group By

Collect rows into groups and then summarize those rows

```
SELECT billingCountry, count(total), min(total),
       max(total), avg(total), sum(total)
  FROM invoices
  GROUP BY BillingCountry
  ORDER BY 2 desc
```

Having

Places restrictions on the rows returned by group by clause

```
SELECT billingCountry, count(total), min(total),
       max(total), avg(total), sum(total)
    FROM invoices
    GROUP BY BillingCountry
    HAVING max(total) > 10
    ORDER BY 4 desc
```

To choose columns	SELECT	name, MIN(year) AS oldest_work_from
To choose a table	FROM	artworks
To join a table	JOIN	artists
	ON	artworks.artist_id = artists.id
To filter records	WHERE	title != “The Mona Lisa”
To group records	GROUP BY	name
To filter groups	HAVING	MIN (year) < 1700
To sort output	ORDER BY	MIN(year);

https://learnsql.com/blog/sql-query-basic-elements/

Creating Records: Insert

```
INSERT INTO tableName
     ( columnName1, columnName2, …, columnNameN)
VALUES
    (value1, value2, …, valueN)
```

```
INSERT INTO artists (ArtistId,Name)
VALUES (99999, 'Rockstar');
```

Modifying Records: Update

```
UPDATE tableName
    SET columnName = 'value'
        {, columnName = 'value' }
    WHERE predicate

UPDATE artists
  SET Name='RockStar’
  WHERE artistID=99999
```

Removing Records: Delete

```
TRUNCATE TABLE tableName

DELETE FROM tableName
    WHERE predicate

DELETE FROM artists
    WHERE name= 'RockStar'
```